| Methods | En.Sum | En.QA | En.MC | En.Dia | Code.D | Math.F | R.PK | R.Num | R.KV | Avg. |
|---|---|---|---|---|---|---|---|---|---|---|
| *Qwen2-7B* | 23.80 | 14.92 | 54.59 | 8.50 | 28.17 | 19.71 | 28.81 | 28.64 | 19.00 | 25.13 |
| NTK | 18.73 | 15.34 | 41.28 | **7.50** | 24.87 | **27.71** | 99.15 | 97.46 | 59.80 | 43.54 |
| SelfExtend | 3.76 | 4.44 | 20.09 | 5.00 | 8.12 | 2.29 | 0.00 | 0.00 | 0.00 | 4.86 |
| StreamingLLM | 19.60 | 13.61 | 48.03 | 3.50 | 27.92 | 19.43 | 5.08 | 5.08 | 2.40 | 16.07 |
| InfLLM | 19.65 | 15.71 | 46.29 | **7.50** | 27.41 | 24.00 | 70.34 | 72.20 | 5.40 | 32.06 |
| **TokenSelect** | **22.62** | **18.86** | **54.31** | **7.50** | **30.20** | 21.71 | **100.00** | **100.00** | **86.60** | **49.08** |
| *Llama-3-8B* | 24.70 | 15.50 | 44.10 | 7.50 | 27.92 | 21.70 | 8.50 | 7.80 | 6.20 | 18.21 |
| NTK | 6.40 | 0.40 | 0.00 | 0.00 | 0.50 | 2.60 | 0.00 | 0.00 | 0.00 | 1.10 |
| SelfExtend | 14.70 | 8.60 | 19.70 | 0.00 | 0.00 | 22.60 | **100.00** | **100.00** | 0.20 | 29.53 |
| StreamingLLM | 20.40 | 14.30 | 40.60 | 5.00 | **28.43** | 21.40 | 8.50 | 8.30 | 0.40 | 16.37 |
| InfLLM | 24.30 | 19.50 | 43.70 | **10.50** | 27.41 | 23.70 | **100.00** | 99.00 | 5.00 | 39.23 |
| **TokenSelect** | **26.99** | **21.32** | **45.85** | 8.00 | 27.41 | **28.29** | **100.00** | 97.29 | **48.40** | **43.90** |
| *Yi-1.5-6B* | 18.78 | 10.48 | 39.74 | 5.00 | 29.95 | 16.00 | 5.08 | 5.08 | 0.00 | 14.45 |
| NTK | 4.66 | 0.58 | 0.87 | 0.00 | 0.00 | 1.43 | 0.00 | 0.00 | 0.00 | 0.83 |
| SelfExtend | 5.62 | 1.07 | 1.31 | 0.00 | 0.00 | 1.14 | 0.00 | 0.00 | 0.00 | 1.01 |
| StreamingLLM | 15.35 | 9.26 | 35.81 | 5.00 | 27.41 | 14.29 | 5.08 | 4.92 | 0.00 | 13.01 |
| InfLLM | 16.98 | 8.93 | 34.06 | 3.00 | 27.41 | 16.86 | **100.00** | 96.61 | 0.00 | 33.76 |
| **TokenSelect** | **21.13** | **12.32** | **40.61** | **5.50** | **30.71** | **20.86** | **100.00** | **99.83** | 0.00 | **36.77** |

Table 1: Comparison of different methods with different origin models on InfiniteBench.

| Methods | 4K | 8K | 16K | 32K | 64K | 128K | Avg. |
|---|---|---|---|---|---|---|---|
| *Qwen2-7B* | 90.74 | 84.03 | 80.87 | 79.44 | 74.37 | 64.13 | 78.93 |
| StreamingLLM | 94.41 | 54.59 | 33.54 | 22.40 | 15.38 | 10.88 | 38.53 |
| InfLLM (2K+512) | 52.85 | 36.09 | 29.36 | 23.52 | 18.81 | 18.29 | 29.82 |
| InfLLM (4K+4K) | 55.22 | 52.10 | 40.53 | 29.77 | 21.56 | 18.64 | 36.30 |
| **Ours (2K+512)** | 94.11 | 81.81 | 68.68 | 60.62 | 51.81 | 42.75 | 66.63 |
| **Ours (4K+4K)** | **94.42** | **90.22** | **82.06** | **70.40** | **59.66** | **54.28** | **75.17** |
| *Llama-3-8B* | 93.79 | 90.23 | 0.09 | 0.00 | 0.00 | 0.00 | 30.69 |
| StreamingLLM | 93.68 | 54.48 | 33.77 | 20.35 | 14.88 | 11.47 | 38.11 |
| InfLLM (2K+512) | 79.79 | 52.43 | 40.12 | 33.60 | 25.68 | 23.39 | 42.50 |
| InfLLM (4K+4K) | 93.79 | 86.11 | 64.33 | 45.39 | 33.13 | 27.81 | 58.43 |
| **Ours (2K+512)** | 93.73 | 82.92 | **71.92** | **65.38** | **59.35** | 33.39 | **67.78** |
| **Ours (4K+4K)** | **93.88** | **90.29** | 70.13 | 57.72 | 48.36 | **39.38** | 66.63 |
| *Yi-1.5-6B* | 73.12 | 9.09 | 0.37 | 0.01 | 0.00 | 0.01 | 13.77 |
| StreamingLLM | 72.10 | 33.03 | 21.69 | 15.39 | 12.58 | 12.61 | 27.90 |
| InfLLM (2K+512) | 59.66 | 36.77 | 27.41 | 24.49 | 21.49 | 21.17 | 31.83 |
| InfLLM (4K+4K) | 74.81 | 52.57 | 27.65 | 22.83 | 20.19 | 19.48 | 36.26 |
| **Ours (2K+512)** | **75.93** | **59.55** | **49.69** | **42.36** | **34.68** | **31.36** | **48.93** |

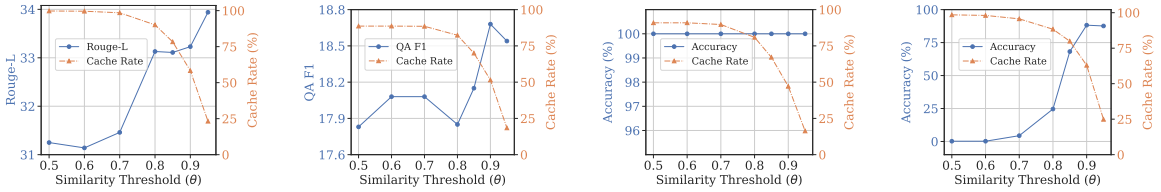
Table 2: Performance comparison on RULER.

StreamLLM, we set $n_{\text{local}} = 4\text{K}$ . For InfLLM, we set $k = 4\text{K}, n_{\text{local}} = 4\text{K}$. For our *TokenSelect*, we set $k = 2\text{K}, n_{\text{local}} = 512$ to demonstrate our token-level KV Cache selection allows us to achieve better performance with a smaller token budget. Due to the need to demonstrate the method under different $n_{\text{local}}$ and $k$, we denote the specific token budgets in the form of $k + n_{\text{local}}$ if they differ from the aforementioned settings. For InfiniteBench and LongBench, we set the threshold $\theta$ of the Selection Cache to 0.9. We use NVIDIA A100 to conduct all experiments. When inferencing sequences over 1M tokens, we additionally employee tensor parallelism, which is transparent to our *TokenSelect*.

### 6.2 Performance Comparisons

**InfiniteBench.** As shown in Table 1, our *TokenSelect* achieves significantly superior overall performance on InfiniteBench compared to all baseline methods, even though *TokenSelect* uses the smallest token budget (<3K). The fact that it significantly outperforms the original models demonstrates *TokenSelect*'s strong length extrapolation capability. We analyze that this is due to our adoption of a fine-grained KV Cache selection strategy, while considering the equal contribution of each head to selection, which ensures that we can select most critical tokens. Observing the performance of other methods, we find that RoPE interpolation methods (NTK, SelfExtend) generally perform poorly unless used on specially trained models such as `Qwen2-7B-Instruct`. The sparse attention method StreamingLLM, based on fixed sparse patterns, can guarantee some of the model's capabilities, but due to discarding a large amount of long-context information, it performs poorly on retrieval-related tasks (R.PK, R.Num, R.KV). The block-level selection method InfLLM can retain more long-context information compared to StreamingLLM. However, due to its sub-optimal block-level selection, it results in lower performance on most tasks compared to *TokenSelect*, even though we set a larger token budget for InfLLM. It is worth noting that `Yi-1.5-6B` does not perform normally on the R.KV task, as it is unable to correctly recite strings like the UUID.

**RULER.** To further demonstrate the capability of *TokenSelect*, we conduct evaluation on the more challenging long-context benchmark RULER. Considering the increased difficulty of RULER and its substantial computational requirements, we include only comparable baseline methods. As shown in Table 2, our *TokenSelect* maintains significantly superior overall performance compared to other long-context inference methods. For all models, *TokenSelect* achieves length extrapolation while preserving the model's original capabilities, benefiting from our efficient utilization of the model's limited context length. Notably, due to the constraints of




(a) LongBench-GovReport. (b) InfiniteBench-En.QA. (c) InfiniteBench-R.PK. (d) InfiniteBench-R.KV.

Figure 6: Performance and Cache Rate with different threshold $\theta$ of the Selection Cache on `Qwen2-7B-Instruct`.

| Methods | En.QA | En.MC | Code.D | R.PK | R.Num | R.KV |
|---|---|---|---|---|---|---|
| *Llama-3-8B-Instruct-262k* | | | | | | |
| *SDPA (128K)* | 9.10 | 68.00 | 19.00 | 100.00 | 100.00 | 17.50 |
| *SDPA (262K)* | 12.40 | 67.30 | 22.10 | 100.00 | 100.00 | 14.40 |
| StreamingLLM (2K+512) | 6.00 | 66.00 | 18.50 | 5.00 | 5.00 | 1.00 |
| SnapKV (2K+512) | 11.80 | 67.00 | 18.00 | 100.00 | 100.00 | 0.50 |
| InfLLM (2K+512) | 7.00 | 37.00 | 20.50 | 100.00 | 100.00 | 0.50 |
| InfiniGen (2K+512) | 7.30 | 57.50 | 17.50 | 100.00 | 99.50 | 0.00 |
| QUEST (2K+512) | 8.20 | 67.00 | 18.00 | 100.00 | 100.00 | 0.00 |
| RetrievalAttn. (2K+512) | 7.50 | 67.00 | 19.00 | 100.00 | 100.00 | 14.00 |
| MInference w/ static | 8.60 | 43.20 | 20.60 | 92.40 | 96.30 | 0.20 |
| MInference | 12.90 | 65.90 | 22.30 | 100.00 | 100.00 | 12.80 |
| Ours (2k+512) | 9.70 | **68.00** | 19.00 | **100.00** | **100.00** | 20.60 |
| *Llama-3-8B-Instruct* | | | | | | |
| Ours (2k+512) | **21.32** | 45.85 | **27.41** | **100.00** | 97.29 | **48.40** |

Table 3: Performance comparison with methods based-on post-trained models. Baseline performance is referenced from Jiang et al. (2024) and Liu et al. (2024a).

model's context length, *TokenSelect* experiences performance degradation with larger token budgets (4K+4K) on Llama and Yi. However, its performance with smaller token budgets still significantly surpasses other baseline methods.

**Comparing to methods based-on post-trained models.** In Table 3, we present a performance comparison of baseline methods that do not support length extrapolation and must be applied to long-text post-trained models. Our results show that, even compared with models undergoing costly long-text post-training and the methods applied to them, the training-free *TokenSelect* exhibits superior performance on most tasks. These findings further demonstrate the effectiveness of *TokenSelect* in long-context inference and length extrapolation.

### 6.3 Ablation Studies

**Selection functions $\mathcal{S}$.** To compare the performance of different selection functions $\mathcal{S}$ under low token budgets (*i.e.*, token efficiency), we maintain the 2K+512 configuration. From Table 4, we can observe that our proposed head soft vote mechanism performs significantly better across all tasks. This indicates that using the head soft vote mechanism to balance each head's contribution to token selection results can help us avoid the domination of selection by few heads with large attention logits.

| $\mathcal{S}$ | En.QA | En.MC | Code.D | R.PK | R.Num | R.KV |
|---|---|---|---|---|---|---|
| $\mathcal{I}_{\text{topk}}$ | 15.15 | 45.85 | 28.43 | **100.00** | 98.47 | 16.60 |
| $\mathcal{I}_{\text{head-vote}}$ | 17.01 | 45.85 | 28.68 | **100.00** | **100.00** | 22.40 |
| $\mathcal{I}_{\text{head-soft-vote}}$ | **18.86** | **54.31** | **30.20** | **100.00** | **100.00** | **86.60** |

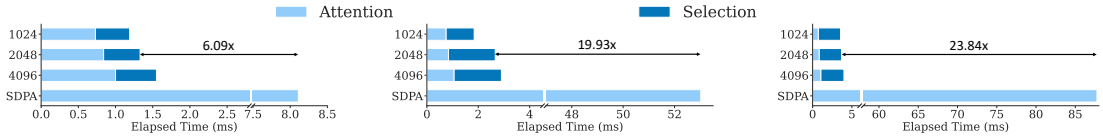
Table 4: Ablation study of the Selection Function $\mathcal{S}$ on InfiniteBench using `Qwen2-7B-Instruct`.

| $k$ | En.Sum | En.QA | En.Mc | Math.F | R.Num | R.KV |
|---|---|---|---|---|---|---|
| 128 | 21.23 | 10.46 | 41.48 | 18.00 | 100.00 | 13.40 |
| 256 | 22.01 | 11.66 | 41.92 | 19.71 | 100.00 | 20.00 |
| 512 | 21.60 | 13.31 | 40.17 | 21.71 | 100.00 | 45.60 |
| 1K | 21.35 | 15.13 | 44.10 | 21.71 | 100.00 | 73.00 |
| 2K | 22.62 | 18.86 | 54.31 | 21.71 | 100.00 | 86.60 |
| 4K | 24.09 | 21.11 | 51.53 | 21.71 | 100.00 | **88.00** |
| 8K | 25.32 | 22.93 | 58.52 | 23.71 | 100.00 | 85.40 |
| 16K | **26.54** | **23.04** | **62.88** | **28.16** | **100.00** | 72.00 |

Table 5: Performance vs. Number of selected tokens $k$ on InfiniteBench using `Qwen2-7B-Instruct`.

**Similarity threshold of the Selection Cache $\theta$.** *Fig.* 6 shows that the Selection Cache hit rate increases significantly as the similarity threshold $\theta$ decreases, converging around $\theta = 0.5$. This suggests potential for further acceleration of *TokenSelect*'s decode stage by reducing $\theta$. Performance sensitivity to $\theta$ varies across tasks. While most tasks exhibit slight performance degradation with decreasing $\theta$, and R.PK in InfiniteBench shows no degradation, more challenging retrieval tasks like R.KV demonstrate significant performance deterioration. This indicates higher dynamicity requirements for token selection in these tasks. Owing to the limited generation lengths in current long-context inference benchmarks, we cannot yet precisely quantify the end-to-end speedup provided by the Selection Cache. Nonetheless, for a 7B-parameter model operating on 128K-token sequences, each cache hit reduces per-step latency by approximately 0.5 ms. For more detailed performance comparisons under different $\theta$, see Table 9 of Appendix I.

**Number of selected tokens $k$.** As shown in Table 5, we fix $n_{\text{local}}$ to a small value (512) to compare the performance when selecting different numbers of tokens. First, we observe that even selecting a very small number of tokens (*e.g.*, 128, 256), our




(a) KV Cache length: 128K. (b) KV Cache length: 512K. (c) KV Cache length: 1M.

Figure 7: Computation time v.s. KV Cache lengths for single chunk prefill step using `Qwen2-7B-Instruct`. The vertical axis represents the number of attended tokens. SDPA denotes full attention by Flashinfer (chunk size: 512).

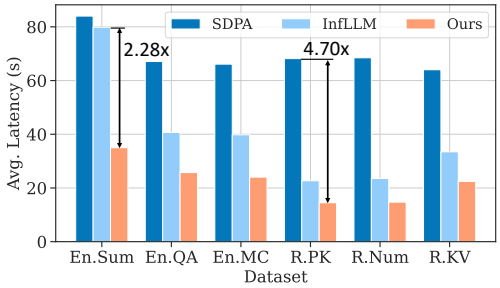



Figure 8: End to end latency per sample with different methods on InfiniteBench using `Qwen2-7B-Instruct`.

*TokenSelect* still demonstrates very comparable performance. Then, as $k$ increases, the effectiveness of *TokenSelect* further improves, indicating that more moderately critical tokens also contribute to the retention of long-context information. Finally, we find that when $k$ is set to larger values (*e.g.*, 16K), our *TokenSelect* shows significant improvements in most tasks, further advancing the performance landscape of long-context inference methods.

### 6.4 Efficiency Comparisons

**Efficiency of selective sparse attention.** *Fig.* 7 demonstrates the significant acceleration of attention computation achieved by *TokenSelect* during long-context inference. With a KV Cache length of 1M, *TokenSelect* can provide up to $23.84\times$ speedup compared to FlashInfer, which is the inference kernel library we based on. This substantial improvement is attributed to our efficient kernel design.

**End-to-end efficiency.** *Fig.* 8 compares the end-to-end latency of *TokenSelect*, InfLLM, and SDPA across various tasks. *TokenSelect* significantly accelerates long-context inference in real-world scenarios, achieving a maximum speedup of $4.70\times$ over SDPA and $2.28\times$ over the state-of-the-art long-context inference method while also delivering superior overall performance.

## 7 Conclusion

In this paper, we introduces *TokenSelect*, a training-free approach for efficient long-context inference and length extrapolation. *TokenSelect* addresses the two major challenges faced by LLMs in processing long texts: the context length limitation from pre-training and the computational complexity of attention. This is achieved through a novel token-level selective sparse attention mechanism. Experimental results demonstrate that *TokenSelect* can achieve up to $23.84\times$ speedup in attention computation and up to $2.28\times$ acceleration in end-to-end inference latency, while exhibiting superior performance across multiple long-context benchmarks.

## 8 Limitations

Our approach has inherent limitations that present opportunities for future work. A primary limitation of our method is that its training-free design—a significant advantage—acts as a double-edged sword, as its absolute performance is inherently tied to the quality of the underlying LLMs. Although our experiments demonstrate robustness of *TokenSelect* across various LLMs, some inherent shortcomings—such as the misrecognition of UUID strings by `Yi-1.5-6B-Chat`—indicate that certain issues may still require training to resolve. Moreover, while our method currently achieves state-of-the-art performance in long-context inference, recent long-text post-training techniques in the LLM community have shown impressive performance; notably, our *TokenSelect* is orthogonal to these approaches and can be employed during inference to trade a slight performance drop for significant efficiency gains. Finally, although our method achieves state-of-the-art efficiency improvements in long-context inference, the task remains inherently resource-intensive. For instance, even with a 8B-parameter model, complex benchmarks (e.g., RULER) can require approximately $8\times$A100 GPUs for nearly one day of runtime, and the computational cost is expected to increase substantially for larger models. We hope that our work, together with the community's advances in model design, algorithm development, and infrastructure optimization, will help pave the way for further mitigating these computational challenges.